

# Incremental (Delta) Data Loading

Using Azure Data Factory, Azure SQL Database & Azure Data Lake Storage Gen2

*Project Documentation*

Watermark-based Incremental Load Pipeline

# 1. Project Overview

This document explains the design and implementation of an incremental (delta) data loading solution built using Azure Data Factory (ADF). The pipeline identifies and extracts only new or changed records from a source Azure SQL Database table since the last pipeline run, and loads them into Azure Data Lake Storage Gen2 (ADLS Gen2). This avoids the cost and time of reloading the entire source table on every run.

The technique used here is the classic “watermark” pattern, which is one of the most common approaches to incremental loading in ADF.

## 1.1 Objective

- Load only newly inserted or modified rows from the source table on each pipeline run, instead of the full table.
- Track the last successfully loaded timestamp (the “watermark”) in a control table.
- Automatically update the watermark after every successful load so that the next run picks up only the newer data.

## 1.2 Architecture Components

| Component                    | Purpose                                                                                                                                                                                                                |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Azure SQL Database           | Source system containing the ‘person’ table (business data) and a ‘watermarktable’ (control table) that stores the last watermark value per table.                                                                     |
| Azure Data Lake Storage Gen2 | Sink / destination for the incrementally extracted data, organized in an ‘output’ container.                                                                                                                           |
| Azure Data Factory           | Orchestrates the pipeline: Lookup activities read the old and new watermark values, Copy Data activity moves only the delta rows, and a Stored Procedure activity updates the watermark table after a successful load. |
| Linked Services              | Connections from ADF to the Azure SQL Database and Azure Data Lake Storage Gen2.                                                                                                                                       |

## 1.3 High-Level Data Flow

1. Lookup activity “Lookup1olddata” reads the last stored watermark value (last successful load time) from the watermark control table.
2. Lookup activity “Lookup1newdata” queries the source table for the current maximum ‘LastModifytime’ value — this becomes the new watermark.
3. Copy Data activity selects only the rows from the source table where LastModifytime is greater than the old watermark and less than or equal to the new watermark, and copies them to the ADLS Gen2 sink as a CSV file.
4. Stored Procedure activity updates the watermark table with the new watermark value, so the next pipeline run starts from where this run left off.

## 2. Environment Setup

### 2.1 Linked Services in Data Factory

Two linked services were initially created in Azure Data Factory — one pointing to Azure Blob Storage and one to the Azure SQL Database. These linked services define the connection information ADF uses to talk to each data store.

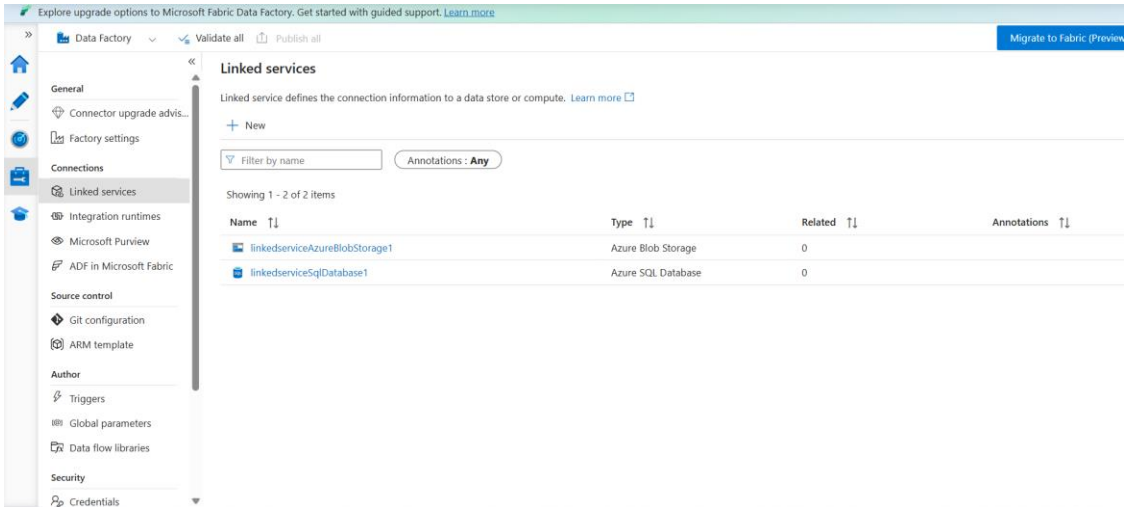


Figure 1: Linked services list in Data Factory — Azure Blob Storage and Azure SQL Database connections.

### 2.2 Storage Account Containers

The target storage account (storenans66) contains two containers: the default '\$logs' container and an 'output' container. The 'output' container is where the incrementally copied data files will be written by the Copy Data activity.

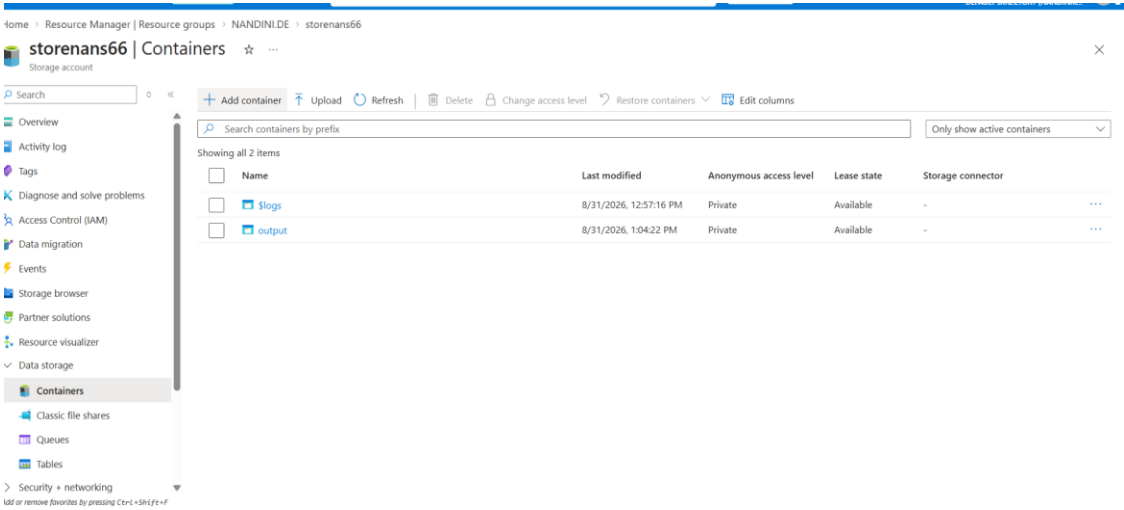


Figure 2: Storage account containers, including the 'output' container used as the sink for incremental loads.

### 2.3 Preparing the Control (Watermark) Table

A stored procedure named 'write\_watermarktable' was created in the Azure SQL Database. It accepts the last modified timestamp and table name as parameters, and updates the watermark value for that table in the 'watermarktable' after every successful incremental load.

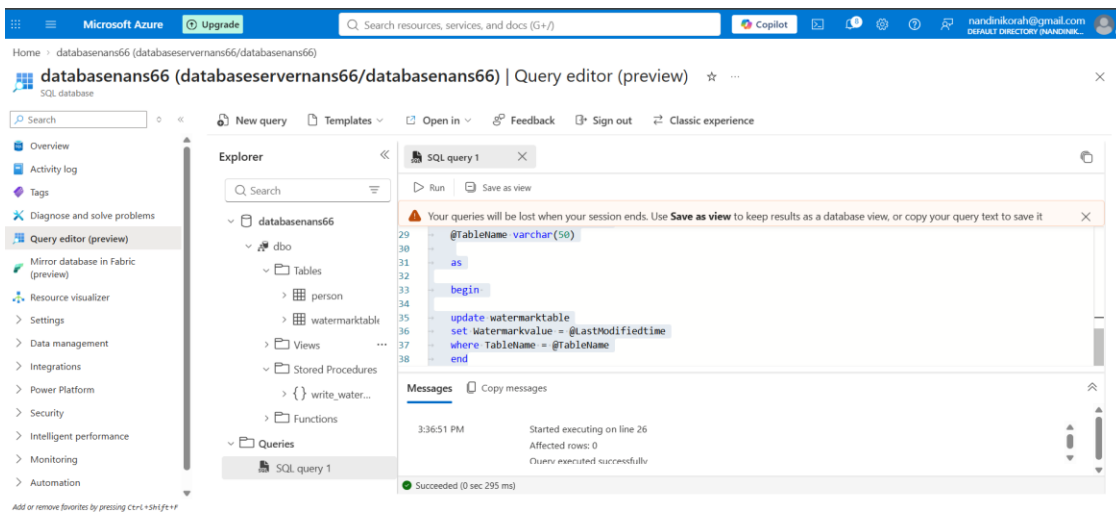


Figure 3: Stored procedure 'write\_watermarktable' — updates Watermarkvalue in watermarktable for the given TableName.

## 2.4 Preparing the Source Table

The 'person' table was created in the Azure SQL Database with PersonID, Name and LastModifytime columns, and seeded with the initial five records. The LastModifytime column is the change-tracking column that the incremental load logic relies on.

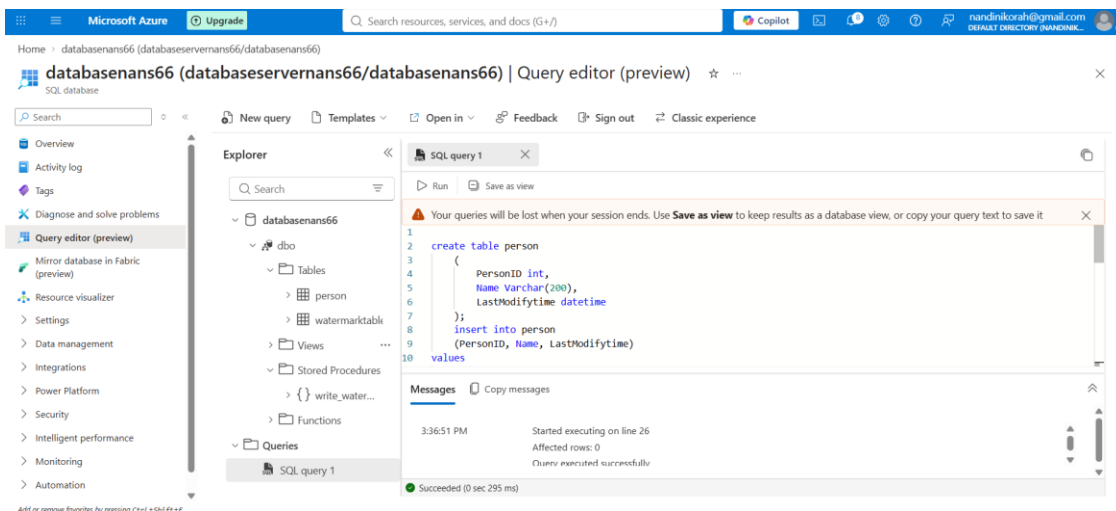


Figure 4: Creating the 'person' source table and inserting the initial batch of records.

## 2.5 Adding the Data Lake Linked Service

A second linked service, 'AzureDataLakeStorage1', of type Azure Data Lake Storage Gen2, was added alongside the existing Azure SQL Database linked service. This will be used as the sink connection for the Copy Data activity.

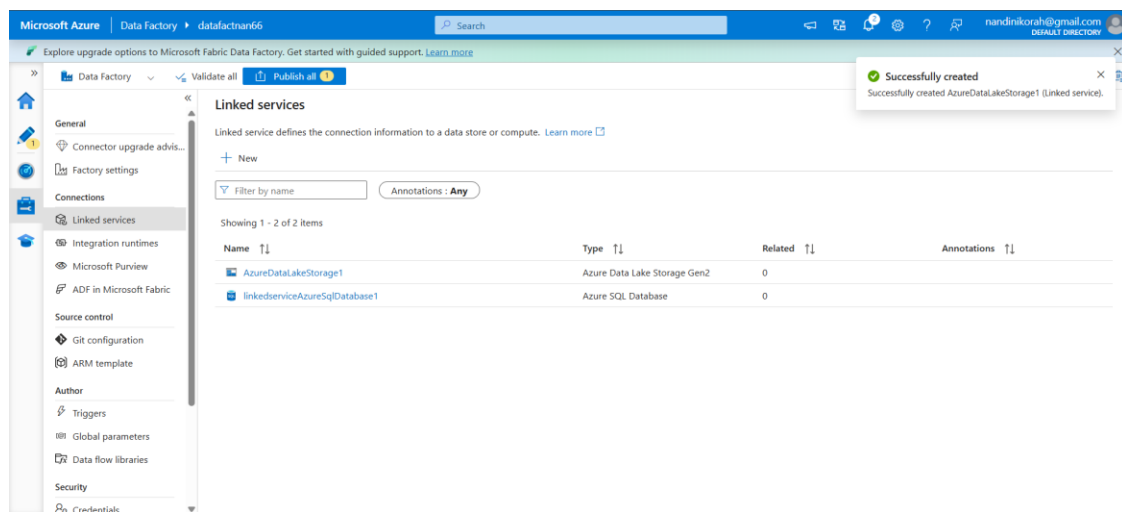


Figure 5: Linked services after adding 'AzureDataLakeStorage1' (ADLS Gen2) next to the Azure SQL Database linked service.

## 3. Building the Incremental Load Pipeline

### 3.1 Creating the Lookup Dataset

A new dataset pointing to Azure SQL Database was created to be used as the source dataset for the Lookup activities that read the old and new watermark values.

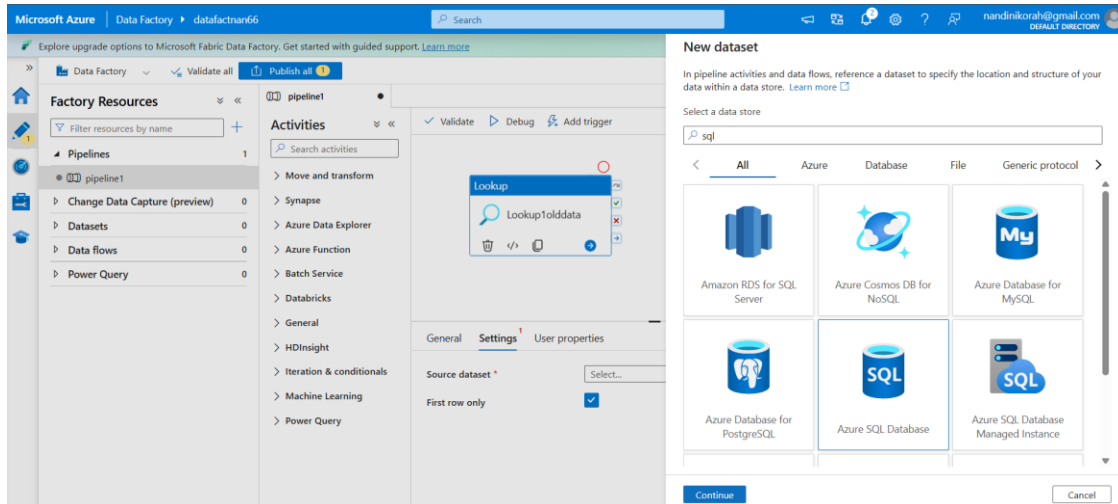


Figure 6: Creating a new dataset and choosing Azure SQL Database as the data store for the Lookup activity.

### 3.2 Lookup Activity – Old Watermark

The first Lookup activity, 'Lookup1olddata', queries the watermarktable for the row where TableName = 'person' and returns the previously stored Watermarkvalue. On the very first run this value was seeded as 2000-01-01, guaranteeing that all existing rows are picked up in the first load.

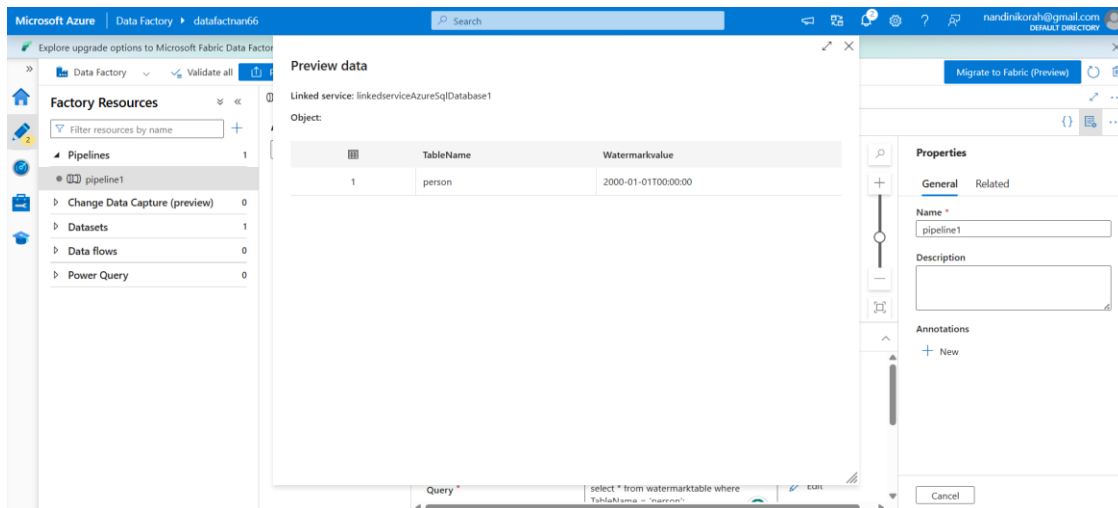


Figure 7: Preview data of Lookup1olddata — returns the stored watermark (2000-01-01) for the 'person' table.

### 3.3 Lookup Activity – New Watermark

The second Lookup activity, 'Lookup1newdata', runs a query against the source 'person' table to fetch MAX(LastModifytime), aliased as NewWatermarkvalue. This represents the newest change timestamp currently present in the source system.

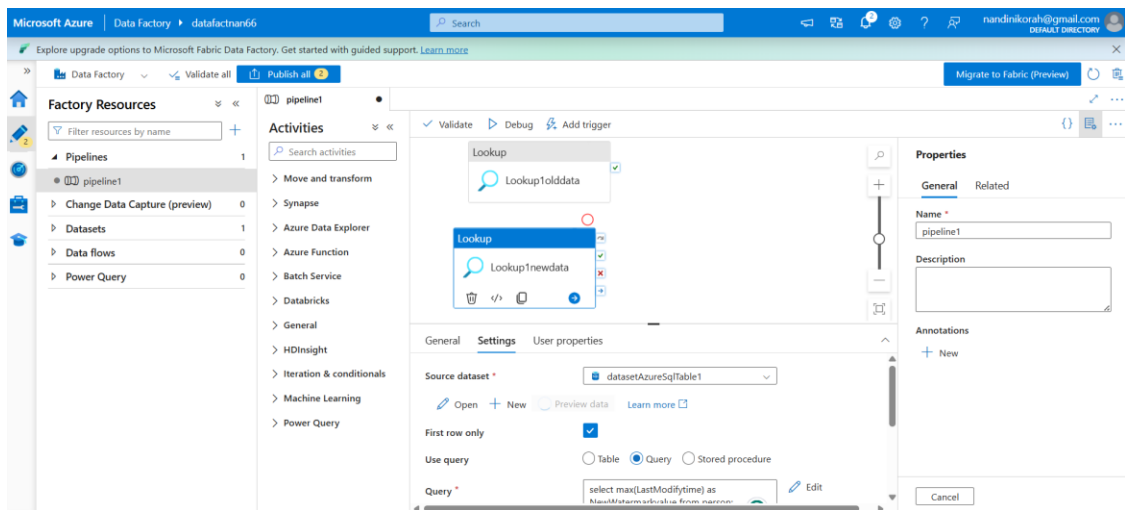


Figure 8: Pipeline canvas showing both Lookup activities — Lookup1olddata and Lookup1newdata — with the new-watermark query configured.

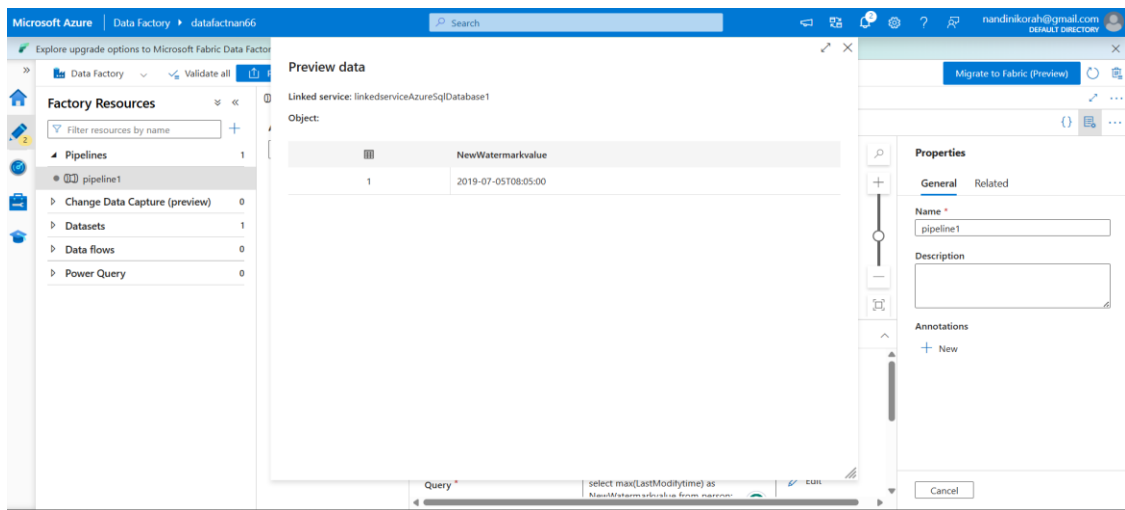


Figure 9: Preview data of Lookup1newdata — returns the current maximum LastModifytime (2019-07-05) from the 'person' table.

### 3.4 Copy Data Activity – Filtering the Delta Rows

The Copy Data activity's source query uses the outputs of both Lookup activities to filter only the rows whose LastModifytime falls strictly after the old watermark and up to the new watermark. This is the core of the incremental extraction logic:

```
SELECT * FROM person WHERE LastModifytime >
'@{activity('Lookup1olddata').output.firstRow.Watermarkvalue}' AND LastModifytime <=
'@{activity('Lookup1newdata').output.firstRow.NewWatermarkvalue}'
```

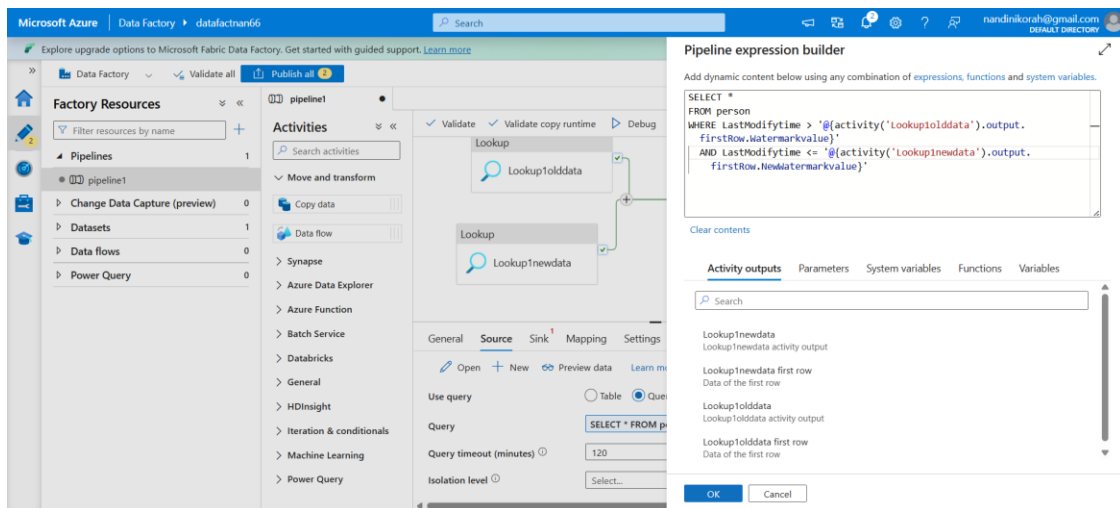


Figure 10: Pipeline expression builder — the dynamic SQL query used by the Copy Data source, referencing both Lookup activity outputs.

### 3.5 Copy Data Activity – Sink Configuration

The sink of the Copy Data activity is a new dataset (datasetgen2) pointing to the AzureDataLakeStorage1 linked service, writing to the 'output' file path in the storage account, with the schema imported from the connected store.

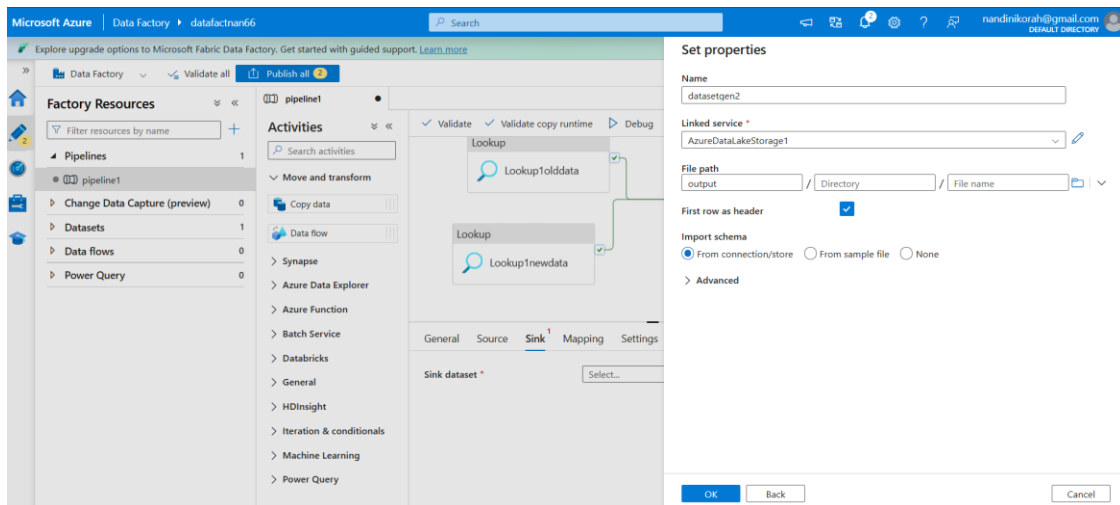


Figure 11: Sink dataset settings for the Copy Data activity — AzureDataLakeStorage1 linked service, output folder path.

### 3.6 Stored Procedure Activity – Updating the Watermark

After the Copy Data activity succeeds, a Stored Procedure activity calls [dbo].[write\_watermarktable] to persist the new watermark value, so that the next pipeline run only picks up data newer than this run's cut-off.

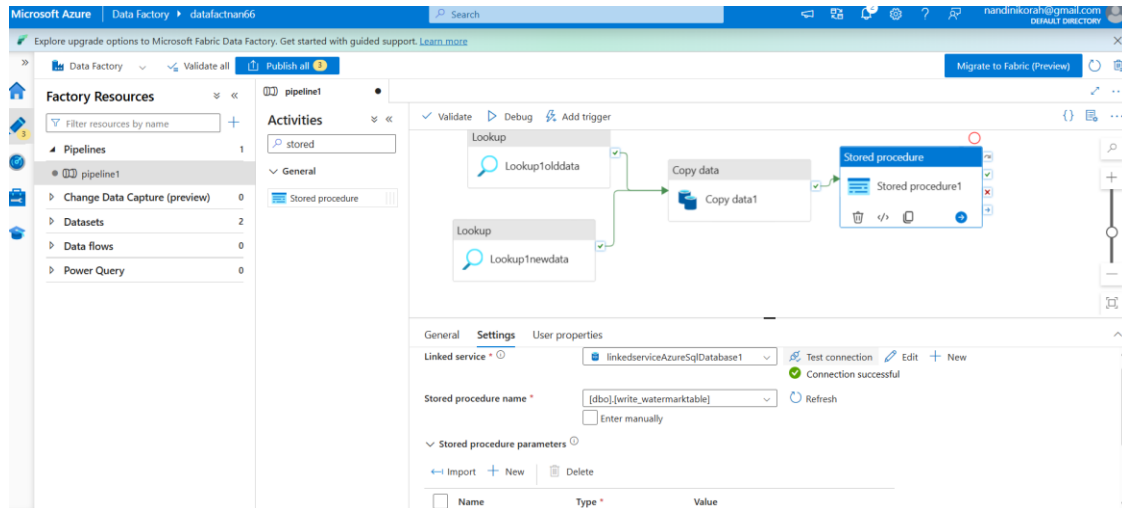


Figure 12: Stored Procedure activity settings — calling write\_watermarktable with a successful connection test.

### 3.7 Binding the Stored Procedure Parameters

The stored procedure's two parameters are mapped dynamically: LastModifiedtime is bound to the output of Lookup1newdata (the new watermark), and TableName is bound to the output of Lookup1olddata (which carries the table name from the control table).

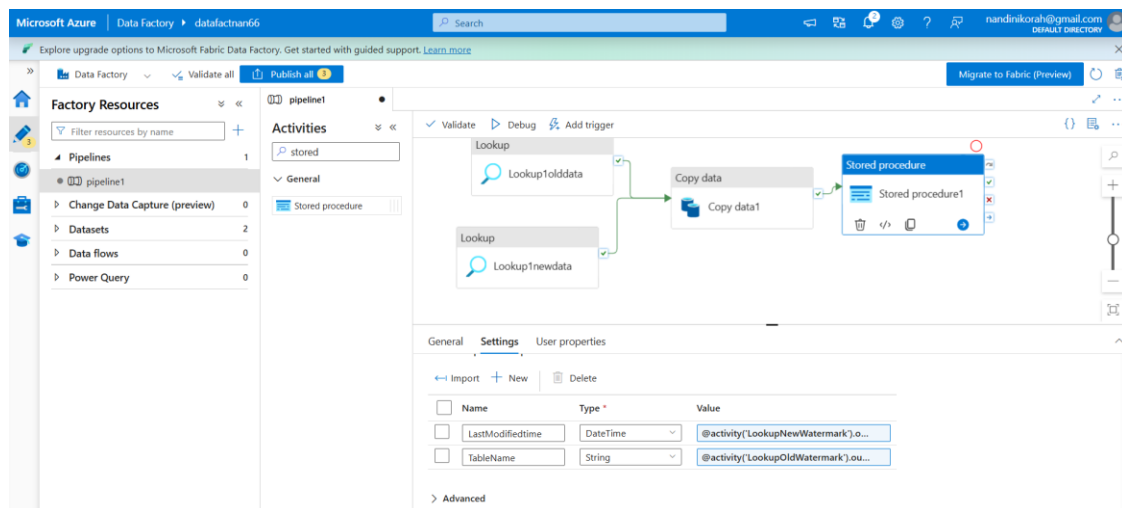


Figure 13: Stored procedure parameters — LastModifiedtime and TableName mapped to the two Lookup activity outputs.

### 3.8 Full Pipeline Layout

The completed pipeline consists of four activities connected in sequence: the two parallel Lookup activities feed into the Copy Data activity, which on success triggers the Stored Procedure activity to update the watermark.

## 4. Validating and Running the Pipeline

### 4.1 Pipeline Validation

Before running the pipeline, it was validated using the Validate option in ADF, confirming there were no configuration errors.

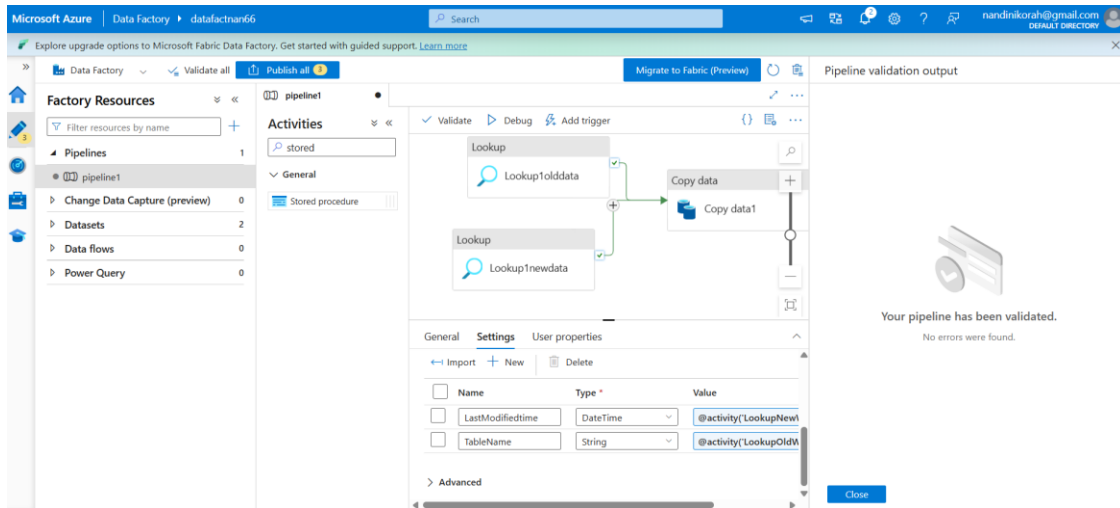


Figure 14: Pipeline validation output — “Your pipeline has been validated. No errors were found.”

## 4.2 First Debug Run — Initial Full Load

The pipeline was run in Debug mode. Since the initial watermark was 2000-01-01, all five existing records in the ‘person’ table qualified as “new” and were copied to the sink. All four activities completed successfully.

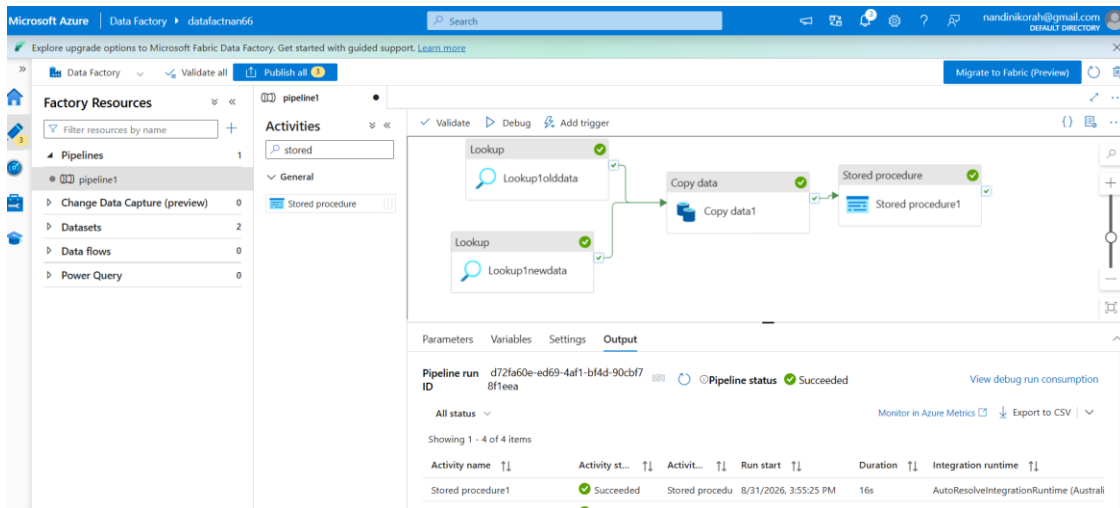


Figure 15: Debug run output — all four activities (two Lookups, Copy data, Stored procedure) succeeded.

## 4.3 Verifying the First Output File

The resulting CSV file in the ‘output’ container contains the five original person records, confirming the first (baseline) load worked correctly.

Microsoft Azure | Upgrade

Search resources, services, and docs (G+/I)

Copilot

nandnikorah@gmail.com  
DEFAULT DIRECTORY (NANDINI...

Home > Resource Manager | Resource groups > NANDINI.DE

output ...

Container

Search

+ Add Directory

Overview

Diagnose and solve problems

Access Control (IAM)

Settings

Authentication methc

Search blobs by

Showing all 1 items

☒ Name

☒ data\_47

data\_47c6c56a-cbb4-40d7-9f9c-237a8633c364\_c0498298-8904-4382-a2da-ad40f831d4b0.csv

Save Discard Download Refresh Delete

Overview Versions Edit Generate SAS

|   | PersonID | Name   | LastModifytime              |
|---|----------|--------|-----------------------------|
| 1 | 1        | "Tiya" | 2019-07-01 00:55:00.0000000 |
| 2 | 2        | "Miya" | 2019-07-02 05:24:00.0000000 |
| 3 | 3        | "Ziya" | 2019-07-03 02:36:00.0000000 |
| 4 | 4        | "Miya" | 2019-07-04 03:28:00.0000000 |
| 5 | 5        | "Iiya" | 2019-07-05 08:05:00.0000000 |

Csv Preview

Add or remove favorites by pressing Ctrl+Shift+F

Figure 16: First output CSV file in ADLS Gen2 — contains the initial five rows from the 'person' table.

## 5. Testing the Incremental Behaviour

To prove that the pipeline only loads new/changed data on subsequent runs, two new records were inserted into the source 'person' table with a LastModifytime later than the previous watermark.

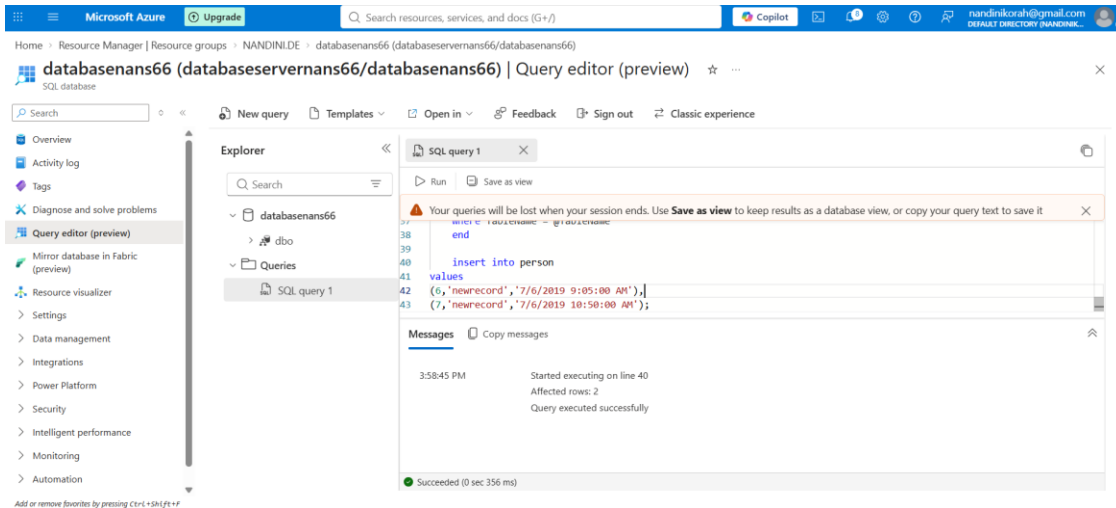


Figure 17: Inserting two new records (PersonID 6 and 7) into the 'person' table to simulate new source data.

### 5.1 Second Pipeline Run

The pipeline was executed again. The Copy Data activity's monitoring details confirm that only 2 rows (60 bytes) were read from the source and 2 rows (116 bytes) were written to the sink — not the full table — proving the incremental filter worked as intended.

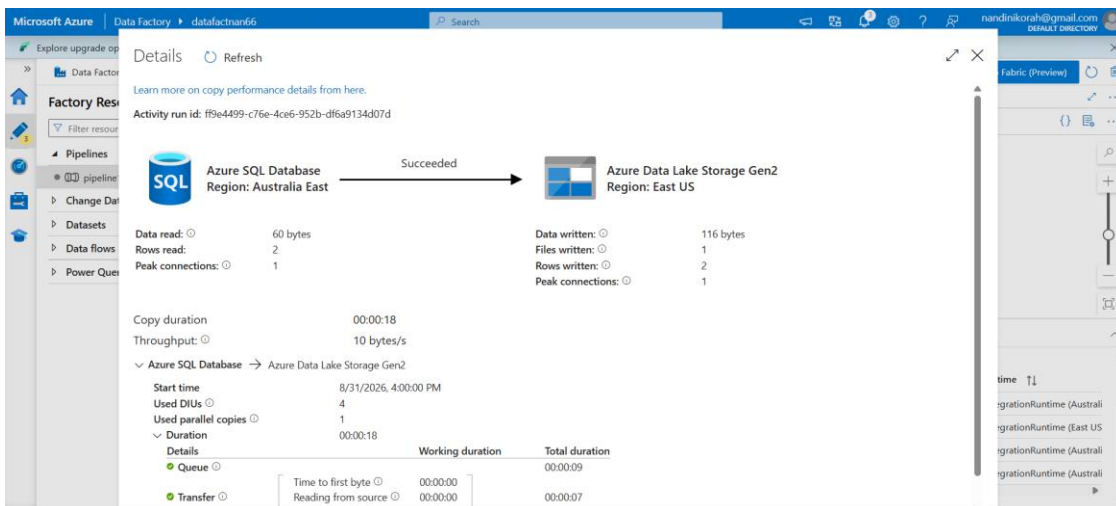


Figure 18: Copy activity execution details for the second run — 2 rows read from Azure SQL Database and 2 rows written to ADLS Gen2.

### 5.2 Verifying the Second Output File

A new CSV file was created in the output container containing only the two newly inserted records, confirming that the earlier five records were not reloaded.

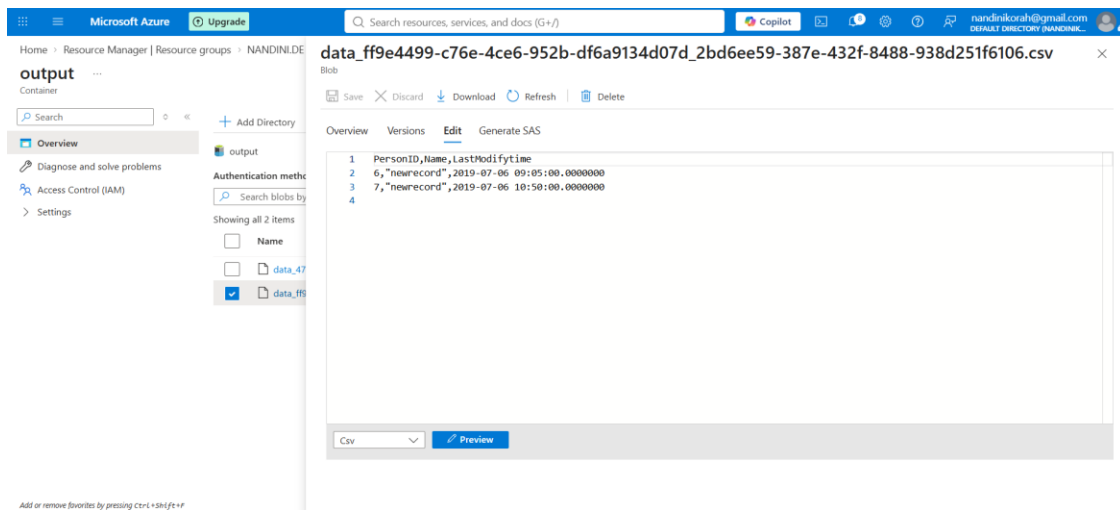


Figure 19: Second output CSV file — contains only the two new 'newrecord' rows (PersonID 6 and 7).

### 5.3 Final Container State

The 'output' container now holds two separate files: one from the initial full load and one from the incremental run, demonstrating that each pipeline execution appends only the delta since the last watermark.

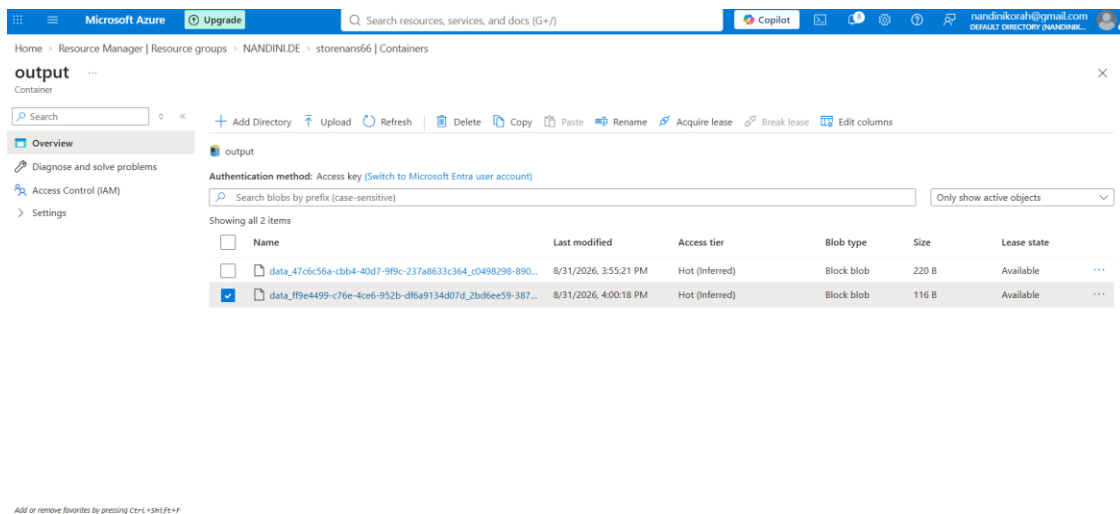


Figure 20: Output container showing both generated files — the baseline load and the incremental load.

## 6. Conclusion

This project demonstrates a complete, working incremental (delta) loading pipeline in Azure Data Factory using the watermark pattern:

- A control table (watermarktable) tracks the last successfully processed timestamp per source table.
- Two Lookup activities determine the old watermark and the current maximum timestamp in the source.
- A dynamically parameterized Copy Data activity extracts only the rows that changed between those two watermark values.
- A Stored Procedure activity advances the watermark after a successful load, ensuring idempotent, incremental behaviour on every subsequent run.

This pattern significantly reduces data movement cost and pipeline run time compared to a full reload, and is one of the standard incremental ETL/ELT design patterns used in real-world Azure Data Factory implementations.